Name: PRANJAL SWARUP BHATNAGAR Reg. No.: RA2512703010007

Course: 21CSC202J - Operating Systems

Unit – 2 : Session – 14 : SLO – 2 | Readers-Writers Problem (Reader Priority) — Missing Code

Answers

TODO 1: pthread_mutex_lock(&mutex);

TODO 2: sem_wait(&wrt); (first reader locks out writers)

TODO 3: pthread_mutex_unlock(&mutex);

TODO 4: pthread_mutex_lock(&mutex);

TODO 5: sem_post(&wrt); (last reader lets writers in)

TODO 6: pthread_mutex_unlock(&mutex);

TODO 7: sem_wait(&wrt);

TODO 8: sem_post(&wrt);

TODO 9: sem_init(&wrt, 0, 1); pthread_mutex_init(&mutex, NULL);

TODO 10: sem_destroy(&wrt); pthread_mutex_destroy(&mutex);

Completed program

```
#include <stdio.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

sem_t wrt;
pthread_mutex_t mutex;
int readcount = 0;
int shared_data = 0;

void* reader(void* arg) {
    int id = *((int*)arg);
    pthread_mutex_lock(&mutex);                /* TODO 1 */
    readcount++;
    if (readcount == 1)
        sem_wait(&wrt);                        /* TODO 2 */
    pthread_mutex_unlock(&mutex);              /* TODO 3 */
    printf("Reader %d is reading: %d\n", id, shared_data);
    sleep(1);
    pthread_mutex_lock(&mutex);                /* TODO 4 */
    readcount--;
    if (readcount == 0)
        sem_post(&wrt);                        /* TODO 5 */
    pthread_mutex_unlock(&mutex);              /* TODO 6 */
    return NULL;
}

void* writer(void* arg) {
    int id = *((int*)arg);
    sem_wait(&wrt);                            /* TODO 7 */
    shared_data++;
    printf("Writer %d is writing: %d\n", id, shared_data);
    sleep(2);
    sem_post(&wrt);                            /* TODO 8 */
    return NULL;
}
```

```
int main() {
    pthread_t r[5], w[2];
    int i;
    int reader_ids[5] = {1, 2, 3, 4, 5};
    int writer_ids[2] = {1, 2};
    sem_init(&wrt, 0, 1);                       /* TODO 9 */
    pthread_mutex_init(&mutex, NULL);
    for (i = 0; i < 5; i++)
        pthread_create(&r[i], NULL, reader, &reader_ids[i]);
    for (i = 0; i < 2; i++)
        pthread_create(&w[i], NULL, writer, &writer_ids[i]);
    for (i = 0; i < 5; i++)
        pthread_join(r[i], NULL);
    for (i = 0; i < 2; i++)
        pthread_join(w[i], NULL);
    sem_destroy(&wrt);                          /* TODO 10 */
    pthread_mutex_destroy(&mutex);
    return 0;
}
```

How it works

1. readcount tracks active readers; mutex protects readcount.
2. The first reader waits on wrt (blocking writers); the last reader posts wrt.
3. Writers take wrt exclusively, so a writer never overlaps a reader or another writer.
4. Readers have priority: while any reader is active, new readers enter and writers wait.